You are transporting some boxes through a tunnel, where each box is a parallelepiped, and is characterized by its length, width and height.

The height of the tunnel 41 feet and the width can be assumed to be infinite. A box can be carried through the tunnel only if its height is strictly less than the tunnel's height. Find the volume of each box that can be successfully transported to the other end of the tunnel.

Note: Boxes cannot be rotated.

Input Format

The first line contains a single integer n, denoting the number of boxes.

n lines follow with three integers on each separated by single spaces - length , width and height which are length, width and height in feet of the i-th box.

Constraints
$1 \leq n \leq 100$
$1 \leq$ length , width , height ≤ 100

Output Format

For every box from the input which has a height lesser than 41 feet, print its volume in a separate line.

Sample Input 0

```
4
5 5 5
1 2 40
10 5 41
7 2 42
```

Sample Output 0

```
125
80
```

Explanation 0

The first box is really low, only 5 feet tall, so it can pass through the tunnel and its volume is 5 x 5 x 5 = 125.

The second box is sufficiently low, its volume is 1 x 2 x 4= = 80.

The third box is exactly 41 feet tall, so it cannot pass. The same can be said about the fourth box.

```c
#include<stdio.h>
struct Box
{
    int l, w, h;
};
int volume(struct Box box)
{
    return box.l*box.w*box.h;
};
int lower(struct Box box, int maxh)
{
    return box.h<maxh;
};
int main()
{
    int n;
    scanf("%d",&n);
    struct Box boxes[100];
    for(int i=0; i<n; i++)
    {
        scanf("%d %d %d",&boxes[i].l,&boxes[i].w,&boxes[i].h);
    }
    for(int i=0; i<n; i++)
        if(lower(boxes[i],41))
        printf("%d\n",volume(boxes[i]));

return 0;
}
```

	Input	Expected	Got	
✓	4 5 5 5 1 2 40 10 5 41 7 2 42	125 80	125 80	✓

Passed all tests! ✓

You are given n triangles, specifically, their sides a , b and c . Print them in the same style but sorted by their areas from the smallest one to the largest one. It is guaranteed that all the areas are different.

The best way to calculate a volume of the triangle with sides a, b and c is Heron's formula:

S = Ö p * (p – a) * (p - b) * (p – c) where p = (a + b + c) / 2.

Input Format

First line of each test file contains a single integer n. n lines follow with a , b and c on each separated by single spaces.

Constraints

$1 \le n \le 100$
$1 \le a , b , c \le 70$
a + b > c , a + c > b and b + c > a

Output Format

Print exactly n lines. On each line print 3 integers separated by single spaces, which are a , b and c of the corresponding triangle.

Sample Input 0

```
3
7 24 25
5 12 13
3 4 5
```

Sample Output 0

```
3 4 5
5 12 13
7 24 25
```

Explanation 0

The square of the first triangle is 84.
The square of the second triangle is 30.
The square of the third triangle is 6.
So the sorted order is the reverse one.

```c
#include<stdio.h>
#include<stdlib.h>
struct tri
{
    int a,b,c;
};
int squ(struct tri t)
{
    int a=t.a, b=t.b, c=t.c;
    return (a+b+c)*(a+b-c)*(-a+b+c);
}
void sort(struct tri*a, int n)
{
    for(int i=0; i<n; i++)
    {
        for(int j= i+1; j<n; j++)\
        {
            if(squ (a[i]) > squ (a[j]))
            {
            struct tri temp=a[i];
            a[i] = a[j];
            a[j] = temp;
            }
        }
    }
}
int main()
{
    int n;
    scanf("%d",&n);
    struct tri a[100];
    for( int i=0; i<n; i++)
    {
        scanf("%d %d %d",&a[i].a,&a[i].b,&a[i].c);
    }
    sort(a,n);
    for(int i=0; i<n; i++)
    {
        printf("%d %d %d\n",a[i].a,a[i].b,a[i].c);
    }
    return 0;
}
```

	Input	Expected	Got	
✓	3 7 24 25 5 12 13 3 4 5	3 4 5 5 12 13 7 24 25	3 4 5 5 12 13 7 24 25	✓

Passed all tests! ✓